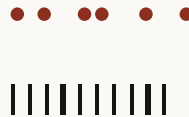


AULA 09

Quatro jeitos de controlar o tempo



cada um resolve o problema do anterior — e traz um novo!

2026.2 · quarta, 09/09

O nosso Pong é um destes quatro

```
dt = clock.tick(60) / 1000  
inputs(dt);  update(dt);  draw()
```

Robert Nystrom descreve quatro maneiras de escrever o laço de um jogo.

Vamos passar pelas quatro, na ordem em que a indústria passou.

Em cada uma: o que ela resolve, e onde ela quebra.

Capítulo Game Loop, gameprogrammingpatterns.com

Run, run as fast as you can

corra o mais rápido que puder

```
while True:  
    inputs(); update(); draw()    # e nada mais
```

O laço gira o mais rápido que a máquina permitir. Sem freio nenhum.

Foi assim nos fliperamas e nos PCs de 1981: toda máquina era igual, e o programador sabia exatamente quantos quadros teria por segundo.

Simples, direto, sem uma linha sobrando.

O que acontece quando
a máquina seguinte é mais rápida?

A velocidade do jogo é a velocidade da máquina

Mais quadros por segundo, mais vezes o `x += 5` acontece: o jogo acelera.

Os PCs ficaram rápidos e os jogos antigos ficaram injogáveis.

O remendo da indústria foi de hardware: o botão TURBO, que servia para DESACELERAR o processador.

DEMONSTRAÇÃO · `07_demo_maquinas_diferentes.py` – tecla 0, depois teclas 1 e 3

Take a little nap

tire um cochilo

```
while True:
    inputs(); update(); draw()
    clock.tick(60)      # durma o que sobrou do quadro
```

O trabalho levou 4 ms? Durma 12. Sempre 16 ms por volta.

Uma linha, e o jogo passa a ter a mesma velocidade em qualquer máquina.

É onde o nosso Pong estava até a semana passada.

E se o trabalho de um quadro
levar mais de 16 ms?

A máquina é fraca, ou a cena ficou pesada.

O cochilo não volta no tempo

O trabalho levou 50 ms. O tick devolve 51, não 16.

Ele consegue esperar, nunca adiantar.

Na máquina fraca o jogo não trava: ele roda em câmera lenta.
Um segundo de jogo passa a durar dois segundos de relógio.

Cada jogador tem um jogo mais lento ou mais rápido conforme o computador.

One small step, one giant leap

um passinho, um passo enorme

```
dt = clock.tick() / 1000      # quanto tempo passou
update(dt)
bola_x += bola_vx * dt        # px por SEGUNDO
```

Se o quadro demorou o dobro, a bola anda o dobro. O jogo acompanha o relógio.

Máquina fraca desenha menos vezes, mas a partida dura o mesmo tanto.

É o que o nosso Pong faz hoje — e onde a maioria dos tutoriais para.

E se um quadro demorar
muito mais que os outros?

Um engasgo de 250 ms: o sistema operacional atrapalhou, um arquivo foi carregado.

O passo cresce junto — e a física salta

	passo da bola	atravessa a raquete?
quadro normal	5 px	não
engasgo de 250 ms	75 px	sim

Nenhum quadro viu a bola dentro da raquete: tunneling.

DEMONSTRAÇÃO · 08_pong_tunelamento.py — segure T

Duas máquinas, a mesma jogada.
A bola faz o mesmo caminho?

Agora sem engasgo nenhum — só duas máquinas diferentes.

Não faz — e nem precisa de engasgo

tecla 2	máquina idêntica, 1/60	diferença zero, sempre
tecla 1	máquina estável de 30 fps	diverge, e nada oscilou
tecla 4	máquina real, irregular	diverge muito

A esquerda é a jogada que o designer definiu. A direita, o que cada máquina faz dela.

DEMONSTRAÇÃO · 09_demo_nao_determinismo.py — teclas 2, 1 e 4

Entra 2 px ou entra 7 px

A bola está a 3 px da parede, a 300 px/s.

Passo de 1/60: anda 5 px, entra 2. Passo de 1/30: anda 10 px, entra 7.

Nos dois casos você a expulsa para a borda — e o excesso é jogado fora.

O quanto foi jogado fora depende do passo. Numa superfície curva, isso muda o ângulo de saída, e a diferença se multiplica a cada quique.

Três situações em que não serve

multiplayer

duas máquinas simulam a mesma partida e mostram a bola em lugares diferentes

replay

gravar as teclas e reproduzir só funciona se a mesma entrada der a mesma saída

teste automatizado

passa numa máquina, falha na outra, e ninguém entende por quê

Como fazer toda máquina
produzir a mesma jogada?

Play catch up

correndo atrás do tempo

Duas perguntas que os anteriores misturavam:
quanto tempo real passou — e de quanto em quanto a física avança.

Movimento em passo fixo, decidido no código.

Desenho quando a máquina conseguir.

Nystrom: é a resposta que ele endossa. Fixed update time step, variable rendering.

O movimento dita o ritmo

Cada ESPAÇO é um passo de 16 ms — sempre o mesmo tanto.

A máquina desenha quando consegue: 16 ms, 21, 33, 50, 80...

Anel = este passo foi desenhado. Ponto simples = ninguém viu.

Segure T e continue apertando ESPAÇO: o movimento não sente o engasgo.

DEMONSTRAÇÃO • 09_demo_movimento_fixo_desenho_variavel.py

Guardar o tempo e gastá-lo em passos

```
PASSO = 1 / 60
```

```
while rodando:
```

```
    acumulador += clock.tick() / 1000
```

```
    inputs()
```

```
    while acumulador >= PASSO:      # gasta em passos iguais
```

```
        update(PASSO)
```

```
        acumulador -= PASSO
```

```
    draw()                          # uma vez, quando der
```

update(PASSO), não update(dt): o dt sai de dentro da física.

Do passo variável ao passo fixo

Três mudanças, todas no laço. Nada muda dentro do update:

```
- dt = clock.tick(60) / 1000      + acumulador += clock.tick() / 1000
- update(dt)                      + while acumulador >= PASSO:
                                  +     update(PASSO)
                                  +     acumulador -= PASSO
```

As velocidades continuam em px por segundo. Muda quem decide o passo.

250 ms viram quinze passos

Antes: um passo de 75 px, e a bola aparecia do outro lado da raquete.

Agora: quinze passos de 5 px, e em algum deles ela está na raquete.

A colisão volta a acontecer porque o passo voltou a ser pequeno.

DEMONSTRAÇÃO · 09_pong_passo_fixo.py – segure T e olhe o contador de passos

E se um passo de física custar
mais do que o passo vale?

A máquina não dá conta de 60 atualizações por segundo.

O acumulador nunca esvazia

Cada volta pede mais passos, que demoram mais, que pedem mais passos.

O jogo congela tentando alcançar o tempo. É a espiral da morte.

```
dt = min(clock.tick() / 1000, 0.25)    # a trava  
# o jogo perde tempo e desacelera – mas volta
```

Perder tempo de jogo é ruim. Congelar é pior.

PERGUNTA

O desenho mostra a bola
onde ela está agora?

Na demo: o quadro 7 foi desenhado 14 ms depois do passo 13.

O desenho mostra o último passo concluído

O que sobrou no acumulador é o tamanho do atraso.

Com passo de 16 ms ninguém percebe; com 250 ms, a demo escancara.

```
alpha = acumulador / PASSO                # 0 a 1  
x_desenho = x_anterior + (x - x_anterior) * alpha
```

Interpolar é desenhar entre o passo anterior e o atual: um passo atrás, sem inventar posição.

Dois jogos com passo fixo,
um de $1/30$ e outro de $1/60$.
Eles produzem a mesma jogada?

Cada um consigo mesmo. Entre si, não.

Rode dez vezes o de 1/30: dá sempre o mesmo resultado. O de 1/60 também.

Mas os dois divergem entre si, pelo mesmo motivo dos 2 px e 7 px.

O tamanho do passo é uma regra do jogo, como a largura da raquete.
Por isso fica cravado no código, igual em todas as máquinas.

A máquina fraca não roda a 30 passos por segundo: roda a 60 e desenha menos vezes.

E se a colisão soubesse o instante exato do contato?

Em vez de perguntar “entrou?” depois do passo.

No Pong, isso bastaria

Avança até o instante do contato, reflete no ponto exato,
gasta o resto do passo já na direção nova.

Com velocidade constante, o tamanho do passo deixa de importar.

E o tunneling morre junto: a varredura testa o caminho, não dois pontos.

“Nós sabemos os pontos, o motor sabe o caminho” — da aula passada.

Porque o Pong é a exceção

aceleração

o cálculo do movimento assume a velocidade constante dentro do passo. Com gravidade ou força, cada tamanho de passo dá uma distância diferente — é o erro de Euler. O Pong não tem aceleração; o jogo de vocês provavelmente terá.

lógica por quadro

o oponente decide a cada volta do laço. A 30 fps decide 30 vezes por segundo; a 120, quatro vezes mais.

generalidade

varredura para círculo e parede é uma equação. Para malhas girando, é um problema caro.

Os quatro jeitos

	resolve	e traz
run as fast as you can	—	velocidade da máquina
take a little nap	velocidade da máquina	câmera lenta na máquina fraca
one small step	câmera lenta	salto · máquinas discordam
play catch up	salto e discordância	atraso no desenho · espiral

O quarto não é o certo: é o que traz os problemas mais fáceis de conviver.

Passo fixo no seu Pong

- 01 Acumulador no laço: `update(PASSO)` em vez de `update(dt)`, com `PASSO = 1/60`.
- 02 A trava `min(dt, 0.25)`. Teste segurando T e observe o contador de passos.
- 03 Desafio: interpolar o desenho. Compare com e sem, usando um passo grande.

A IA pode escrever o laço. Você explica por que ele tem dois `while`s.